

Portfolio Assignment: Write Up

ICT 4305: Object Oriented Methods and Programming

for

Master of Science

Information Technology

Kalika Browder

University of Denver College of Professional Studies

November 13, 2025

Faculty: Nirav Shah, M.S. , MBA

Director: Cathie Wilson, M.S.

Dean: Bobbie Kite, PhD

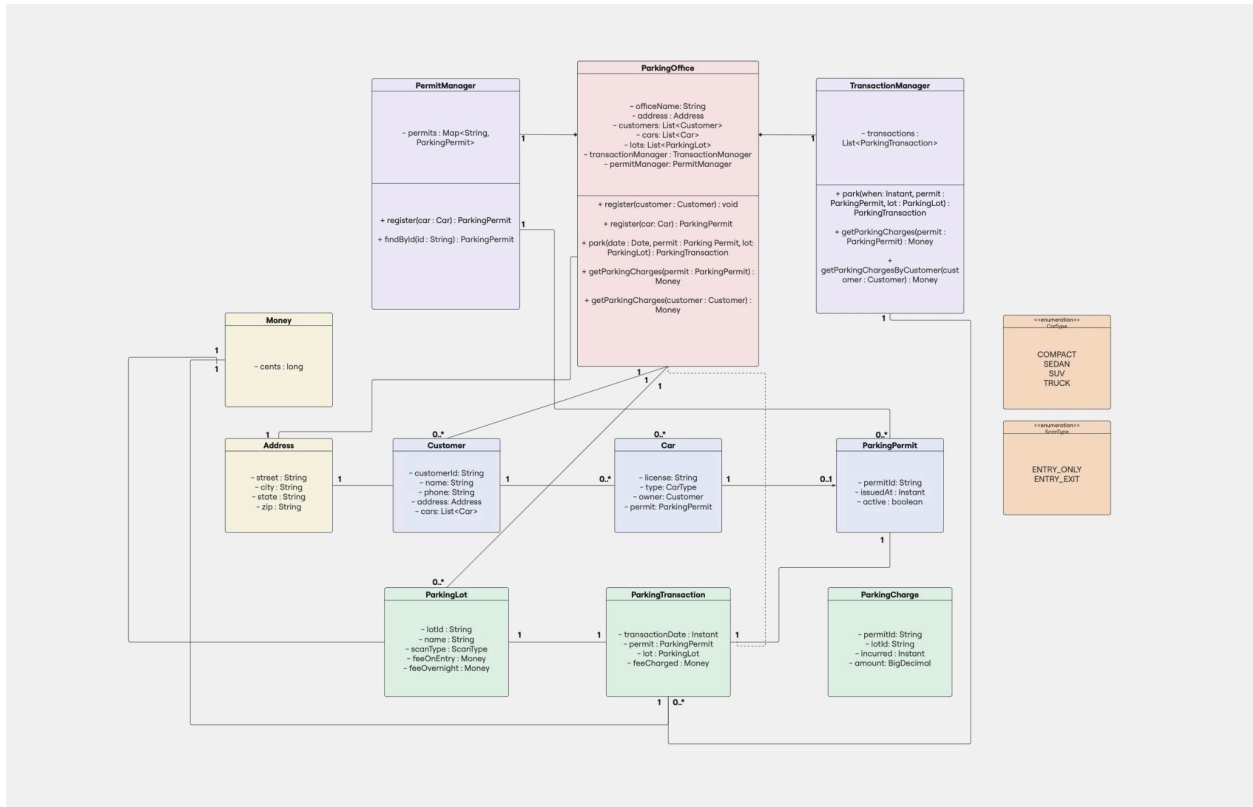
Reflection

Building this parking system throughout the entirety of this course has been absolutely one of the most challenging and simultaneously rewarding experiences that I have had in terms of Java and Object Oriented Programming thus far. What began as a series of smaller exercises, including the designing of the classes, adding attributes, writing tests, and implementing manager classes, ultimately evolved into a fully integrated system that required so much careful coordination to ensure that it functioned across multiple components. This final portfolio gave me the opportunity to bring all of those pieces together, incorporate feedback that I had garnered, and most importantly, debug and refine my work in a way that reinforced nearly everything that I had learned in discussions and coursework.

Project Overview

The goal of the project overall was to construct a complete, object-oriented model of a parking office system, ideally what the University of Denver or a similar corporate campus might use. The system was designed to support registering customers and their vehicles, generating parking permits, recording parking transactions, and calculating parking charges. The structure centers around the `ParkingOffice` class, which acts as the main “hub” of functionality for the system. It also collaborates with two essential manager classes, the first being the `PermitManager` which creates and stores `ParkingPermit` objects. The second was the `TransactionManager` which records parking events and calculates cumulative charges.

Supporting these core classes are several domain entities, including Customer, Car, Address, ParkingLot, ParkingTransaction, ParkingCharge, and Money. Each class models a real-world concept, which allows the resulting system to be both realistic and conceptually coherent to onlookers.



Parking System Class Diagram

The above diagram of the system highlights the key relationships throughout classes, the first being **ParkingOffice**. This maintains a list of customers, cars, and parking lots. It delegates permit creation to **PermitManager** and transaction handling over to **TransactionManager**. **PermitManager** stores a map of permit IDs to **ParkingPermit** objects and then creates new

permits when vehicles are registered. The TransactionManager records parking events as ParkingTransaction objects, stores them, and calculates totals for a permit or for a customer.

The Customer class is associated with one more Car object, while the Car object has a license plate, a type, and is linked to the ParkingPermit object. ParkingPermit is then assigned to a car and valid one year from registration. ParkingTransaction represents a single instance of parking in a particular lot, at a particular time. Lastly, ParkingLot class is used to define a fee structure (such as entry fees), while Money is used to represent currency safely and consistently throughout the system.

This structure of the parking system is designed to follow clean, object-oriented design principles. These include encapsulation, delegation, single-responsibility, and separation of concerns. Each class performs one role, and the system is flexible enough to continue to extend in future assignment iterations, if need be.

Takeaways

The absolute most significant learning experience in this project came from the debugging and integration aspect of the system. Conceptually, designing classes was straightforward, as the challenge came from when these classes began interacting in such a complex network. Seemingly small inconsistencies, such as using Permit instead of ParkingPermit, or missing a getter like getParkingLot() cascaded down into failing tests and broken functionality throughout my system.

The most major lesson that I learned from this process was that debugging is not a setback, but an essential part of the developmental process. Several times, throughout the

project, errors surfaced because of misaligned expectations between tests and implementations, or because method signatures changed in my code and the tests still relied on old behaviors I forgot to update. A lot of these issues forced me to trace relationships between my classes, rethink all of my assumptions, and then also search for mismatched types and missing methods.

I mentioned this in my previous assignment reflections, but the importance of test-driven thinking has been at the forefront of my mind. Because unit tests were required for each class, I had to constantly consider how each component might be validated in the system. When tests failed, and specifically when one of my tests kept consistently failing, they pointed directly to inconsistencies in my code and in the tests themselves. This process helped me better understand proper cohesion between all of the system parts.

Probably the most important learning moment came from when I had to integrate the TransactionManager. Initially, test failures kept indicating that recorded parking transactions were always calculating at \$0.00. Debugging, which took me the better part of four hours, revealed to me that although the transaction object was being created, the system wasn't assigning the lot's entry fee properly. By fixing this and ensuring that the transaction stored the correct fee, I not only resolved the test, but also corrected a real functionality flaw. The moment of relief when all of my tests came out running properly was so rewarding, also!

The most significant tool that I was forced to become comfortable with during this assignment was definitely JUnit. Before this project, I had limited exposure to automated unit testing. I also had a learning curve because before this project, I had limited exposure to

automated unit testing. Learning how to structure tests, assert behavior, and interpret JUnit's output gave me a deeper appreciation for how large, professional systems maintain reliability and also detect regressions in systems.

I was familiar with Terminal, but I feel like this whole process also allowed me to learn how to use command-line Java tooling more efficiently and effectively including cleaning compiled files, running compilation with classpaths, executing JUnit from the console, and structuring a project so that compilation and test execution work smoothly. I am so proud of this! Command-line used to deeply unnerve me, and now I feel comfortable opening up my Terminal window.

All of this was crucial to my insight. If I could restart this project from the beginning, I would keep my codebase more disciplined early on... Several of my debugging challenges came from remnants of older code from previous assignments. This included outdated permit classes, unused attributes, incomplete placeholder methods, etc. These artifacts created confusion when integrating newer components.

I would also adopt strong documentation habits. I went through such a big process to get to my final code. I wish I would have noted that throughout my Java files. I could have noted what I added, what I removed, or even commented out what wasn't working. Instead I kept deleting and moving and coming back to code I had rewritten thousands of times, it felt like. Keeping personal notes during development would have helped me track design decisions moreover and helped me avoid misunderstandings later on.

This system truly has nowhere to go but up. Given additional time, I would expand the project in several ways. The first thing I would do to grow it would be to add expiration handling for permits, that way the system could automatically detect expired permits and disable the parking privileges. Then I would allow for exit scanning and hourly charges, so instead of a fixed entry fee, I could compute fees based on time parked. Something else I was considering might be a persistence layer. I could use it to store all customers, permits, and transactions in a simple database or file, which would then allow me to implement search and reporting features. I could allow for queries like “total revenue per lot”, which would streamline all of the data collected. Creating a small user interface for the interaction with the ParkingOffice class would also be ideal, given the time. Each enhancement of this system would make it more realistic and closer to production quality, which if I had the time would be so fun to see through!

This project was such a good learning experience for me. The ability to blend design, debugging, and testing into one cohesive process pushed me to think critically about how objects interact, how to structure code responsibly, and how to systematically identify and resolve errors. It was a great exercise in critical thinking, and what I could be doing better. The debugging journey, while sometimes tedious, made me a stronger and more confident programmer. The final result, while maybe not perfect, is a fully functioning parking system that showcases assignment needs and lays a solid foundation for my understanding of Java going forward.